

Padaria Adonai

Referência do Código · Versão Java

Sistema de Gestão de Padaria

Trabalho Prático 1 · Algoritmos e Estruturas de Dados

ISUTC · Engenharia Informática e de Telecomunicações

Parte 1 · Como funciona o sistema

Parte 2 · Referência técnica: classes, variáveis e funções

Parte 1 · Como funciona a Padaria Adonai

Uma explicação simples, passo a passo

Uma padaria dentro do computador

Imagina uma padaria de brincar que vive dentro do computador. Lá dentro não há pão a sério, mas o computador lembra-se de **todos os pães** que existem: como se chamam, quanto custam, quantos há na prateleira e até quando podem ser comidos. Também se lembra de **tudo o que foi vendido**.

Este programa é o **ajudante** do dono da padaria. Em vez de escrever tudo num caderno, o dono carrega em botões e o computador faz as contas e arruma tudo.

A porta, o porteiro e a loja

Quando abres o programa, passas por três sítios, sempre pela mesma ordem:



Os três ecrãs do programa. O botão Sair leva-te outra vez para a porta.

- 1. A porta.** Vês uma fotografia de pães com o nome *Padaria Adonai*. Carregas em **Acessar** para entrar.
- 2. O porteiro.** Pergunta quem és e qual é a tua **palavra secreta** (a senha). Se te enganares, ele diz *“Utilizador ou senha inválidos”* e não te deixa passar. Repara que ele não diz *qual* das duas está errada: assim, um espertalhão não consegue adivinhar os nomes dos utilizadores.
- 3. A loja.** À esquerda há uma coluna castanha cheia de botões, um para cada coisa que podes fazer. À direita há duas tabelas: uma com os **pães** (Registos) e outra com as **vendas**.

Em linguagem de programador

Os três sítios são as classes `TelaInicial`, `TelaLogin` e `AplicacaoPadaria`. Nenhuma conhece as outras: quem as liga é o método `novaSessao()` da classe `Main` (no pacote principal), como um guia que te leva de sala em sala. Cada ecrã recebe um *callback* (um `Runnable` ou a interface `TelaLogin.AoAutenticar`) para chamar quando acaba. Todos os ecrãs usam a mesma janela (`JFrame`): cada um troca o seu conteúdo.

O dono e o ajudante

Na padaria trabalham duas pessoas: o **dono** e o **ajudante** (o funcionário). O dono tem **todas as chaves**: pode pôr pães novos, mudar preços e deitar pães fora. O ajudante pode procurar pães, ver as listas e vender, mas não pode mexer no que há na loja.

E há um truque: os botões que o ajudante não pode usar **nem sequer aparecem** para ele. É como se essas portas não existissem. E se, por acaso, alguém encontrasse uma dessas portas,

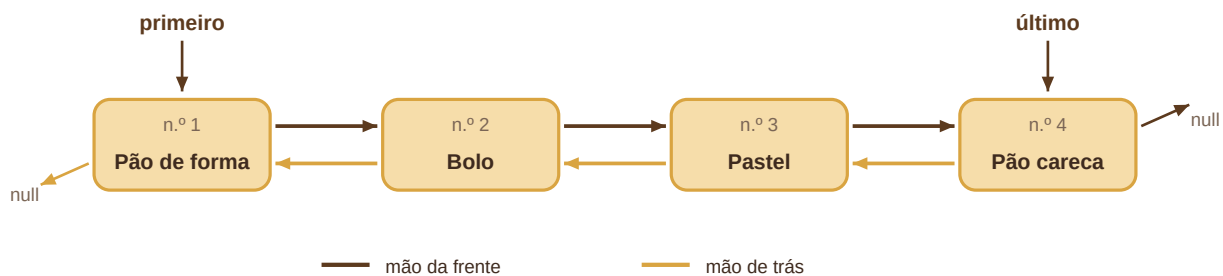
há um segundo guarda lá dentro que diz “Esta operação está reservada ao Dono da Padaria”.

Em linguagem de programador

A lista das operações proibidas ao ajudante é `OPERACOES_RESTRITAS_AO_DONO` (em `SistemaPadaria`). Cada utilizador é um objecto `Utilizador`, e `eDono()` diz se é o dono. O “segundo guarda” é o método `exigeDono()` de `AplicacaoPadaria`.

A fila de pães de mãos dadas

Esta é a parte mais importante. O computador guarda os pães numa **fila**, como meninos numa fila da escola. Cada pão dá **uma mão ao pão da frente** e **outra mão ao pão de trás**.



Cada pão sabe quem está à frente e quem está atrás. As pontas não dão a mão a ninguém (null).

O computador não precisa de se lembrar de onde está cada pão. Só precisa de saber três coisas:

- quem é o **primeiro** da fila;
- quem é o **último** da fila;
- **quantos** pães há.

Para encontrar um pão, começa pelo primeiro e pergunta: “És tu o número 3?” Se não for, segue a mão da frente para o próximo e pergunta outra vez, até o encontrar ou até a fila acabar. A isto chama-se **percorrer** a fila.

Em linguagem de programador

A fila é uma **lista duplamente ligada**: a classe `ListaLigadas`, que segue a interface `InterfaceGeral`. Cada pão fica dentro de um **nó** (`No`) com `getProximo()` (mão da frente) e `getAnterior()` (mão de trás). A lista guarda `primeiro`, `ultimo` e `totalElem`. Quem “anda” pela fila é a variável `actual`: `actual = actual.getProximo()`; . A fila dos pães é a `ListaProdutos`, que **herda** de `ListaLigadas` (extends) e faz tudo com os métodos dela (`adicionaFim`, `pega`, `removePosicao...`).

Chega um pão novo

Quando o dono carrega em **Cadastrar**, abre-se uma janela onde escreve os dados do pão novo. O pão novo vai para o **fim da fila**: o último pão dá-lhe a mão, e o pão novo passa a ser o último.

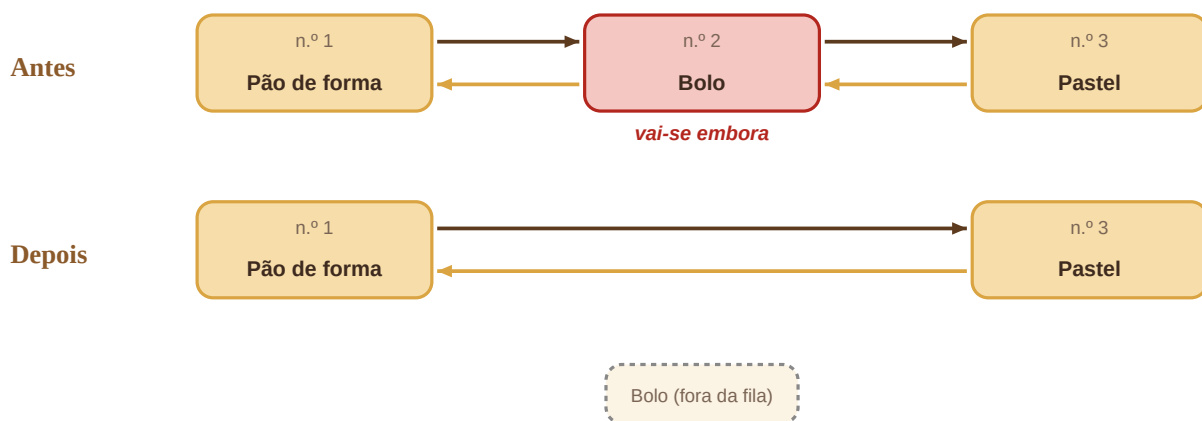
Antes disso, o computador confirma que não há outro pão com o mesmo número, tal como numa equipa de futebol não pode haver dois jogadores com o mesmo número na camisola. Se houver, aparece um aviso, e a janela fica aberta para o dono corrigir sem ter de escrever tudo outra vez.

Em linguagem de programador

`cadastrar(produto)` pergunta à lista `contem(produto)`. Isto funciona porque `Produto.equals` só compara o código: dois produtos com o mesmo número são “o mesmo”. Se não existir, chama `adicionaFim(produto)`.

Um pão vai-se embora

Quando o dono deita um pão fora (**Eliminar**), esse pão sai da fila. Mas a fila não pode ficar partida! Por isso, o pão que estava atrás e o que estava à frente **dão as mãos um ao outro**.



O Bolo sai, e o Pão de forma e o Pastel passam a dar as mãos.

Se quem sai é o primeiro, o segundo passa a ser o primeiro. Se é o último, o penúltimo passa a ser o último. E aqui as duas mãos dão jeito: para saber quem é o novo último, basta seguir a **mão de trás** do último, sem ter de percorrer a fila toda.

Também dá para dizer “*tira o 3.º da fila*”. Para nós a fila começa no 1, mas o computador conta a partir do 0, por isso o 3.º é a posição 2 para ele.

Em linguagem de programador

`eliminarPorPosicao(posicao)` verifica `posicaoValida(posicao - 1)`, guarda o produto com `pega(posicao - 1)` (para dizer o nome a quem eliminou) e chama `removePosicao(posicao - 1)`. Esse método usa `removeInicio()` ou `removeFim()` nas pontas e, no meio, liga os dois vizinhos entre si.

Procurar pães

Há várias maneiras de procurar:

- **Buscar Produto:** pelo número. “*Quero o pão número 3.*”
- **Buscar (1 atributo):** por uma característica. “*Mostra todos os que são Pão.*”
- **Buscar (2 atributos):** por duas ao mesmo tempo. “*Os que são Pão e custam 15.*”

O computador não liga a maiúsculas: “PÃO”, “pão” e “Pão” são a mesma coisa para ele. O que encontra vai para **uma fila nova** (outra `ListaLigadas`), que é a que aparece na tabela.

Mudar um pão

Se o preço do bolo subir, o dono carrega em **Alterar**, diz o número do bolo, e abre-se a janela já preenchida. Ele muda só o que quer e carrega em **Guardar alterações**. É como trocar a etiqueta do preço. O número do pão (o da camisola) nunca muda, e por isso esse campo aparece bloqueado.

Pôr os pães por ordem

Para ver os pães do mais barato para o mais caro, o computador usa um método chamado **bolha** (*bubble sort*). Olha para dois pães vizinhos; se o mais caro estiver à frente, trocam de lugar. Faz isto ao longo da fila, muitas vezes, até ninguém precisar de trocar. Os mais caros vão “subindo” para o fim, como bolhas de ar na água.



Um segredo: o computador ordena uma **cópia** da fila, só para mostrar. A fila verdadeira fica igual.

Em linguagem de programador

`listarOrdenado(atributo, decrescente)` copia os produtos para um array `Produto[]` (com `pega(i)`), ordena o array com bubble sort e devolve uma `ListaLigadas` nova, preenchida com `adicionaFim`.

Vender pão

Chega um cliente e pede 3 pães de forma. Carregas em **Registar Venda**, escreves o número do pão e a quantidade. Antes de vender, o computador confirma três coisas:

1. escreveste números a sério, e a quantidade é maior que zero?
2. esse pão existe?
3. há pães suficientes na prateleira?

Só se as três estiverem certas é que tira os pães da prateleira e escreve a venda no **caderno de vendas**. E o caderno de vendas... é outra fila de mãos dadas! Lá em baixo, o computador soma quantos pães se venderam e quanto dinheiro entrou.

O caderno guarda o preço daquele dia. Se amanhã o pão ficar mais caro, a venda de hoje continua com o preço antigo, como num talão de compras.

Em linguagem de programador

As vendas ficam na `ListaVendas`, que também herda de `ListaLigadas`; cada elemento é um objecto `Venda`. As três verificações estão em `registarVenda()` de `AplicacaoPadaria`, e o stock só é descontado depois de todas passarem.

As cores da padaria

Todas as cores estão guardadas numa só caixa de lápis: castanho de crosta, dourado de pão acabado de sair do forno e creme de miolo. Se trocares um lápis dessa caixa, a padaria inteira muda de cor de uma vez.

Em linguagem de programador

A caixa de lápis é a parte “Tema visual” de `SistemaPadaria`: as constantes `FUNDO`, `PAINEL`, `CRISTA`, `DOURADO...` (objectos `Color`), as fontes `FONTE_*` e o método `aplicarTema()`.

Porque é que tudo desaparece quando fecho?

A padaria escreve tudo num **quadro de giz** (a memória do computador), e não num caderno (um ficheiro). Quando fechas o programa, o quadro é apagado. Na próxima vez, começa outra vez com os 4 pães de exemplo e 3 vendas de exemplo.

Onde está cada coisa

O ficheiro que liga o programa está sozinho no pacote principal. Todos os outros estão no pacote `listas_duplamente_ligadas`, como as salas de uma casa:

Sala	Ficheiro	Para que serve
O interruptor e o guia	<code>principal/Main.java</code>	Liga o programa (<code>main</code>) e leva-te da porta ao porteiro e do porteiro à loja. É este que se corre.
O livro de regras	<code>SistemaPadaria.java</code>	Nome da padaria, moeda, utilizadores, senhas e quem pode fazer o quê.
A caixa de lápis	<code>SistemaPadaria.java</code>	Cores, letras e o aspecto dos botões e tabelas.
As mãos dadas	<code>InterfaceGeral.java</code> , <code>No.java</code> , <code>ListaLigadas.java</code>	A fila de mãos dadas em si. Serve para qualquer coisa: pães, vendas...
O armazém	<code>ListaProdutos.java</code> , <code>ListaVendas.java</code>	A fila dos pães e a fila das vendas.
As etiquetas	<code>Produto.java</code> , <code>Venda.java</code> , <code>Utilizador.java</code>	O que se sabe de cada pão, de cada venda e de cada pessoa.
A loja	<code>TelaInicial.java</code> , <code>TelaLogin.java</code> , <code>AplicacaoPadaria.java</code>	Tudo o que se vê: porta, porteiro, loja e janelas.
A fotografia	<code>src/imagens/fundo_inicial.png</code>	A imagem de pães da porta.
O inspector	<code>test/.../TestesListas.java</code>	Verifica que as filas continuam de mãos dadas depois de cada mudança.

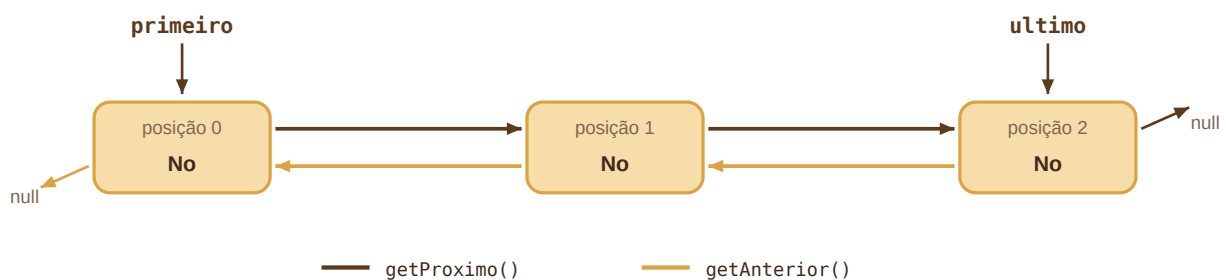
Parte 2 · Referência técnica

Todas as classes, as suas variáveis e o que cada função faz e como

Visão geral

O programa arranca em `principal.Main`, a única classe do pacote principal. Todas as outras estão no pacote `listas_duplamente_ligadas`. `Main.main()` aplica o tema e abre três ecrãs por ordem: `TelaInicial` → `TelaLogin` → `AplicacaoPadaria`. Os produtos e as vendas ficam em `ListaProdutos` e `ListaVendas`, que **herdam** de `ListaLigadas`. A interface lê os dados das janelas, converte os valores, chama um método da lista e volta a desenhar as tabelas. Nada é gravado em disco: cada execução começa com os dados de exemplo.

A lista guarda `Object`: cada nó leva um `Produto` ou uma `Venda`, e quem o lê faz o `cast`, por exemplo (`Produto`) `pega(i)`. As listas da padaria **nunca mexem nas ligações** entre nós: fazem tudo através dos métodos de `ListaLigadas`. As estruturas também não conhecem a interface, e é isso que permite testá-las sozinhas.



Cada nó aponta para o anterior e para o próximo. A lista guarda primeiro, ultimo e totalElem. As posições começam em 0.

Invariantes da lista

Verdadeiras antes e depois de qualquer método público:

- `lista vazia ↔ primeiro == null ↔ ultimo == null ↔ totalElem == 0`
- `primeiro.getAnterior() == null` e `ultimo.getProximo() == null`
- para todo o nó `n` com próximo: `n.getProximo().getAnterior() == n`
- as posições válidas vão de 0 a `totalElem - 1` (`posicaoValida`)

1. A lista duplamente ligada (a base)

Estas três classes são a base do projecto. Todas as outras estruturas usam-nas.

interface IntefaceGeral

As operações que qualquer lista tem de oferecer. ListaLigadas implementa-a, e por isso também ListaProdutos e ListaVendas.

Função	O que faz	Como
adicionaInicio(elemento)	Insere no início.	Ver ListaLigadas.
adicionaPosicao(posicao, elemento)	Insere numa posição.	Ver ListaLigadas.
adicionaFim(elemento)	Insere no fim.	Ver ListaLigadas.
pega(posicao)	Devolve o elemento de uma posição.	Ver ListaLigadas.
removeInicio() / removePosicao(posicao) / removeFim()	Removem um elemento.	Ver ListaLigadas.
contem(elemento)	Diz se o elemento está na lista.	Compara com equals.
tamanho()	Número de elementos.	Devolve totalElem.
posicaoValida(posicao)	Diz se a posição existe.	$0 \leq \text{posicao} < \text{tamanho}()$.
estaVazio()	Diz se a lista está vazia.	$\text{tamanho}() == 0$.

class No

Um nó: um elemento e as ligações aos vizinhos.

Variável	Tipo	Significado
elemento	Object	O que o nó guarda (um Produto, uma Venda...).
anterior	No (ou null)	Nó anterior; null no primeiro nó.
proximo	No (ou null)	Nó seguinte; null no último nó.

Função	O que faz	Como
No(elemento)	Cria um nó solto.	$\text{anterior} = \text{proximo} = \text{null}$.
No(elemento, proximo)	Cria um nó já ligado à frente.	$\text{anterior} = \text{null}$.
No(anterior, elemento, proximo)	Cria um nó ligado aos dois lados.	Guarda os três valores.
getElemento()	Lê o elemento.	Não há setElemento: o elemento de um nó não muda.
getProximo() / setProximo(no)	Lê / muda a mão da frente.	—
getAnterior() / setAnterior(no)	Lê / muda a mão de trás.	—

class ListaLigadas implements InterfaceGeral

A lista duplamente ligada.

Variável	Tipo	Significado
primeiro	No (ou null)	Primeiro nó (cabeça). null quando a lista está vazia.
ultimo	No (ou null)	Último nó (cauda). Permite inserir e remover no fim sem percorrer a lista.
totalElem	int	Número de nós. Atualizado em cada inserção e remoção.

Função	O que faz	Como
ListaLigadas()	Cria uma lista vazia.	primeiro = ultimo = null, totalElem = 0.
adicionaInicio(elemento)	Insere no início.	Cria um No com proximo = primeiro. Se a lista estiver vazia passa a ser primeiro e ultimo; senão primeiro.setAnterior(novo) e primeiro = novo. O(1).
adicionaPosicao(posicao, elemento)	Insere numa posição (0 = início).	IndexOutOfBoundsException fora de 0..totalElem. Posição 0 → adicionaInicio; posição totalElem → adicionaFim; senão liga o novo nó entre pegaNo(posicao - 1) e pegaNo(posicao). O(n).
adicionaFim(elemento)	Insere no fim.	Lista vazia → adicionaInicio. Senão ultimo.setProximo(novo), novo.setAnterior(ultimo) e ultimo = novo. O(1).
pega(posicao)	Devolve o elemento da posição.	IndexOutOfBoundsException se a posição não for válida. Anda posicao vezes a partir de primeiro com getProximo(). O(n).
removeInicio()	Remove o primeiro.	IllegalStateException se vazia. Com 1 elemento: primeiro = ultimo = null. Senão primeiro passa para o seguinte, que fica com anterior = null. O(1).
removePosicao(posicao)	Remove o elemento da posição.	IndexOutOfBoundsException se inválida. Posição 0 → removeInicio; última → removeFim; no meio, pegaNo(posicao) e liga o anterior e o próximo entre si. O(n).
removeFim()	Remove o último.	NullPointerException se vazia. Com 1 elemento: lista fica vazia. Senão ultimo = ultimo.getAnterior() e ultimo.setProximo(null). O(1), graças à mão de trás.
contem(elemento)	Procura um elemento.	Percorre a partir de primeiro e compara com equals. O(n).
tamanho()	Número de elementos.	Devolve totalElem. O(1).
posicaoValida(posicao)	Valida uma posição.	true se $0 \leq \text{posicao} < \text{totalElem}$.
estaVazio()	Lista vazia?	true se totalElem == 0.
pegaNo(posicao) [privado]	Devolve o nó (não o elemento).	Como pega, mas devolve o No. Usado por adicionaPosicao e removePosicao.
concatenar(lista)	Junta outra lista no fim desta.	NullPointerException se a outra estiver vazia. Liga ultimo → lista.primeiro e soma os tamanhos. As duas listas passam a partilhar os nós. O(1).
RemoverTudo(obj)	Remove todas as ocorrências de obj.	NullPointerException se obj for null. Percorre a lista e remove cada nó igual, religando os vizinhos. O(n).
contem(InterfaceGeral lista)	Elementos de outra lista que também estão nesta.	Para cada elemento da outra lista chama contem(elemento); devolve uma ListaLigadas nova. O(n-m).

Função	O que faz	Como
<code>Ocorrencias(IntefaceGeral lista)</code>	Quantas vezes aparece cada elemento.	Devolve um <code>int[]</code> com, para cada elemento da outra lista, o número de nós iguais nesta. $O(n \cdot m)$.

Em linguagem de programador

Três correcções feitas nesta classe ao passar o projecto para Java: (1) `adicionaFim` criava o nó com `new No(elemento, this.ultimo)`, o que ligava o novo último ao antigo e fechava a fila num círculo (o contem nunca terminava). Passou a `new No(elemento)`. (2) `public void ListaLigada()` não era um construtor e passou a `public ListaLigadas()`. (3) `lista.ArrayList` não existe neste projecto e foi trocada por `IntefaceGeral / ListaLigadas`.

2. Os dados da padaria

São os objectos que vão dentro dos nós (o elemento).

class Produto

Um produto. É o elemento de cada nó da `ListaProdutos`.

Variável	Tipo	Significado
<code>codigo</code>	<code>int</code>	Código único do produto. Não tem set: nunca muda.
<code>nome</code>	<code>String</code>	Nome do produto.
<code>categoria</code>	<code>String</code>	Categoria (ex.: Pão, Bolo, Doce).
<code>preco</code>	<code>double</code>	Preço unitário (MT).
<code>quantidade</code>	<code>int</code>	Unidades em stock.
<code>validade</code>	<code>String</code>	Data de validade (dd/mm/aaaa).

Função	O que faz	Como
<code>Produto(codigo, nome, categoria, preco, quantidade, validade)</code>	Cria um produto.	Guarda os seis valores.
<code>getX() / setX(valor)</code>	Lêem e mudam cada atributo.	Só existe <code>getCodigo()</code> , sem <code>setCodigo</code> .
<code>equals(obj)</code>	Compara produtos.	Iguais se tiverem o mesmo código . É isto que deixa <code>contem(produto)</code> detectar códigos repetidos.
<code>hashCode()</code>	Coerente com <code>equals</code> .	Calculado a partir do código.
<code>toString()</code>	Texto do produto.	"código - nome".

enum Produto.Atributo

Os seis atributos do produto, pela ordem em que aparecem na tabela, na janela de produto e nas buscas: CODIGO, NOME, CATEGORIA, PRECO, QUANTIDADE, VALIDADE.

Variável	Tipo	Significado
rotulo	String	Nome amigável (ex.: PRECO → “Preço”).
dica	String (ou null)	Texto ao lado do campo (preço → MT, validade → dd/mm/aaaa).

Função	O que faz	Como
valorEm(produto)	Valor deste atributo num produto.	switch que chama o get certo. Devolve um Comparable (Integer, Double ou String), para as buscas e para a ordenação.
getRotulo() / getDica()	Lêem o rótulo e a dica.	—
toString()	Texto mostrado nas listas de escolha.	Devolve o rótulo (“Preço” e não “PRECO”).

class Venda

Uma venda. É o elemento de cada nó da ListaVendas. Guarda uma cópia do nome e do preço do momento da venda, e não muda depois de criada.

Variável	Tipo	Significado
codigoProduto	int	Código do produto vendido.
nomeProduto	String	Nome do produto no momento da venda.
quantidade	int	Unidades vendidas.
precoUnitario	double	Preço por unidade no momento da venda.
total	double	quantidade × precoUnitario, calculado no construtor.
data	String	Data e hora da venda (dd/MM/yyyy HH:mm).
FORMATO_DATA	DateTimeFormatter	Formato usado em data (constante).

Função	O que faz	Como
Venda(codigoProduto, nomeProduto, quantidade, precoUnitario)	Cria uma venda.	Calcula total e data (LocalDateTime.now()).
getX()	Lêem cada atributo.	Não há set: uma venda não se altera.

class Utilizador

Um utilizador do sistema. É o “perfil” que a `AplicacaoPadaria` recebe depois do login.

Variável	Tipo	Significado
senha	String	Senha (privada: só se compara, nunca se lê).
tipo	String	PERFIL_DONO ou PERFIL_FUNCIONARIO.
nome	String	Nome mostrado na saudação.

Função	O que faz	Como
senhaCorrecta(senha)	Verifica a senha.	iguais entre a senha guardada e a escrita.
eDono()	É o dono?	Compara tipo com PERFIL_DONO.
getTipo() / getNome()	Lêem o tipo e o nome.	—

3. As listas da padaria

As duas herdam de `ListaLigadas` (extends) e só usam os métodos dela: não têm variáveis próprias.

class ListaProdutos extends ListaLigadas

Todos os produtos. As posições que o utilizador vê começam em 1; as de `ListaLigadas` começam em 0. “Listar todos” é a própria lista.

Função	O que faz	Como
cadastrar(produto)	Acrescenta um produto no fim.	<code>IllegalArgumentException</code> se <code>contem(produto)</code> (código repetido); senão <code>adicionaFim(produto)</code> . $O(n)$.
buscarPorCodigo(codigo)	Encontra um produto pelo código.	<code>posicaoDoCodigo</code> ; devolve (<code>Produto</code>) <code>pega(posicao)</code> ou null.
buscarPorUmAtributo(atributo, valor)	Produtos em que um atributo é igual a um valor.	Percorre com <code>pega(i)</code> e usa <code>coincide</code> . Os encontrados vão para uma <code>ListaLigadas</code> nova com <code>adicionaFim</code> .
buscarPorDoisAtributos(a1, v1, a2, v2)	Produtos que cumprem dois atributos.	Mesmo percurso; guarda o produto só se as duas comparações forem verdadeiras (E lógico).
alterarPorCodigo(codigo, nome, categoria, preco, quantidade, validade)	Edita um produto.	<code>buscarPorCodigo</code> (<code>IllegalArgumentException</code> se não existir) e chama os set. Valores null ou "" são ignorados: o campo mantém o valor antigo.
eliminarPorPosicao(posicao)	Remove o produto na posição (1 = primeiro).	<code>IndexOutOfBoundsException</code> se <code>!posicaoValida(posicao - 1)</code> ; guarda <code>pega(posicao - 1)</code> , chama <code>removePosicao(posicao - 1)</code> e devolve o produto removido.
eliminarPorCodigo(codigo)	Remove o produto com esse código.	<code>IllegalStateException</code> se <code>estaVazio()</code> ; <code>IllegalArgumentException</code> se o código não existir; senão <code>pega + removePosicao</code> .
listarPorCritério(atributo, valor)	Produtos que cumprem um critério.	Delega em <code>buscarPorUmAtributo</code> .
listarOrdenado(atributo, decrescente)	Produtos ordenados por um atributo.	Copia para um <code>Produto[]</code> com <code>pega(i)</code> , aplica <code>bubble sort</code> (compara com <code>compareTo</code> sobre <code>atributo.valorEm</code>) e devolve uma <code>ListaLigadas</code> nova. $O(n^2)$. A lista original não muda.

Função	O que faz	Como
<code>posicaoDoCodigo(codigo)</code> [privado]	Posição (a partir de 0) de um código.	Percorre com <code>pega(i)</code> ; -1 se não existir.
<code>coincide(produto, atributo, valor)</code> [privado]	Compara um atributo com um texto.	Os dois lados como texto, sem espaços e em minúsculas ("pão" encontra "Pão").

class `ListaVendas` extends `ListaLigadas`

Todas as vendas, pela ordem em que aconteceram. Só se acrescenta: as vendas não se alteram nem se apagam.

Função	O que faz	Como
<code>registarVenda(codigoProduto, nomeProduto, quantidade, precoUnitario)</code>	Regista uma venda no fim.	Cria uma Venda e chama <code>adicionaFim</code> . Devolve a venda. Não valida stock (isso é feito na interface).
<code>totalVendas()</code>	Receita total.	Percorre com <code>pega(i)</code> somando <code>getTotal()</code> .
<code>totalQuantidade()</code>	Total de unidades vendidas.	Percorre com <code>pega(i)</code> somando <code>getQuantidade()</code> .

4. Arranque, configuração e interface

A interface usa **Swing**. Todos os ecrãs usam a mesma janela (`JFrame root`): cada ecrã troca o conteúdo dela e recebe um *callback* para chamar quando termina, por isso os ecrãs não precisam de se conhecer. Os diálogos são `JDialog` modais.

class principal.`Main`

Ponto de entrada: é a classe que se corre. Está sozinha no pacote `principal`, separada de todas as outras, e só usa classes públicas de `listas_duplamente_ligadas`.

Função	O que faz	Como
<code>main(args)</code>	Início do programa.	<code>SistemaPadaria.aplicarTema()</code> e depois <code>novaSessao()</code> na thread do Swing (<code>invokeLater</code>).
<code>novaSessao()</code> [privado]	Encadeia os três ecrãs.	Cria a janela; define <code>aoAutenticar</code> (abre <code>AplicacaoPadaria</code>) e <code>aoAcessar</code> (abre <code>TelaLogin</code>); mostra <code>TelaInicial</code> . É também o callback do botão Sair, por isso sair recomeça numa janela nova.

class `SistemaPadaria`

Configuração e tema visual, usados por todos os ecrãs.

Configuração

Variável	Tipo	Significado
NOME_PADARIA	String	“Padaria Adonai”. Usado nos títulos e ecrãs.
MOEDA	String	“MT”. Usado em todos os valores monetários.
PERFIL_DONO / PERFIL_FUNCIONARIO	String	Nomes dos perfis; compara-se sempre com estas constantes.
UTILIZADORES	Map<String, Utilizador>	utilizador → Utilizador. Dois: dono / dono123 (Dono da Padaria) e funcionario / func123 (Funcionário).
OPERACOES_RESTRITAS_AO_DONO	Set<String>	Texto dos botões que só o dono vê: Cadastrar, Alterar (por código), Eliminar por posição, Eliminar por código.

Tema visual

Variável	Tipo	Significado
FUNDO, PAINEL, CROSTA, CROSTA_ESCURA, DOURADO, DOURADO_CLARO, TEXTO, TEXTO_SUAVE, BRANCO, ERRO	Color	Paleta de toda a aplicação. As outras classes não têm cores escritas à mão.
FONTE_TITULO, FONTE_SUBTITULO, FONTE_SECCAO	Font	Georgia (títulos).
FONTE_NORMAL, FONTE_NEGRITO, FONTE_PEQUENA	Font	SansSerif (texto).

Função	O que faz	Como
aplicarTema()	Configura o aspecto geral.	Activa o aspecto Metal (o do macOS/Windows ignora as cores dos botões) e define cores e fontes por omissão no UIManager. Chamar uma vez, antes de criar janelas.
botao / botaoSecundario / botaoSidebar(texto)	Criam botões com o estilo da padaria.	Dourado, castanho, ou castanho alinhado à esquerda. Todos usam botaoBase.
botaoBase(...) [privado]	Botão com cor normal e cor com o rato por cima.	Cores, cursor de mão e um MouseListener que troca as cores em mouseEntered/mouseExited.
rotulo(texto, fonte, cor)	Cria um JLabel.	—
configurarTabela(tabela)	Estilo das tabelas.	Cabeçalho castanho, texto centrado, selecção dourada e linhas alternadas (um DefaultTableCellRenderer).
centrarJanela(janela)	Centra a janela no ecrã.	Um pouco acima do meio (altura / 3).
margem(componente, v, h)	Margem interior vazia.	EmptyBorder.

class TelaInicial

Ecrã de abertura: imagem de fundo, nome da padaria e botão “Acessar”.

Variável	Tipo	Significado
CAMINHO_IMAGEM_FUNDO	String	“/imagens/fundo_inicial.png”, procurada no classpath.
LARGURA	int	Largura da janela: 640 px.
ALTURA_IMAGEM	int	Altura da zona da imagem: 320 px.
ALTURA_RODAPE	int	Altura do rodapé com o botão: 90 px.

Função	O que faz	Como
TelaInicial(janela, aoAcessar)	Constrói o ecrã.	Um JPanel que desenha a imagem centrada e os textos (paintComponent); rodapé escuro com o botão. Enter também entra (setDefaultButton). O tamanho mínimo é o da imagem + rodapé.
carregarImagem() [privado]	Lê a imagem.	Primeiro do classpath (getResource); se não estiver lá, de src/imagens. Sem imagem fica só o fundo castanho.
textoComSombra(...) [privado]	Texto legível sobre a imagem.	Desenha o texto duas vezes: uma sombra escura deslocada 2 px e o texto claro por cima.

class TelaLogin

Ecrã de login. Só passa quem estiver em UTILIZADORES.

Variável	Tipo	Significado
AoAutenticar	interface	Callback autenticado(utilizador), chamado depois de um login certo.
LARGURA / ALTURA	int	Tamanho da janela: 420 × 460 px.
aoAutenticar	AoAutenticar	O callback recebido no construtor.
entradaUtilizador	JTextField	Campo do utilizador.
entradaSenha	JPasswordField	Campo da senha (mostra •).
labelErro	JLabel	Mostra “Utilizador ou senha inválidos.”

Função	O que faz	Como
TelaLogin(janela, aoAutenticar)	Constrói o formulário.	Nome da padaria, dois campos, etiqueta de erro e botão “Entrar”. Enter no utilizador salta para a senha; Enter na senha tenta entrar.
tentarLogin() [privado]	Verifica as credenciais.	Procura o utilizador em UTILIZADORES e usa senhaCorrecta. Errado: mostra o erro e limpa a senha. Certo: chama aoAutenticar.autenticado(registo).
campo(...) / adicionar(...) [privados]	Estilo dos campos e arrumação no BoxLayout.	—

class AplicacaoPadaria

Janela principal depois do login: barra de topo, sidebar com as operações e duas abas (Registos / Vendas). Os dados do produto são pedidos numa janela própria.

Variável	Tipo	Significado
LARGURA, ALTURA	int	Tamanho inicial: 1150 × 700 px (mínimo 950 × 600).
lista	ListaProdutos	Todos os produtos.
listaVendas	ListaVendas	Todas as vendas.
root	JFrame	Janela principal.
perfil	Utilizador	O utilizador autenticado.
eDono	boolean	true se o perfil for o dono.
aoTerminarSessao	Runnable	Callback chamado ao sair (recomeça na tela inicial).
abas	JTabbedPane	O controlo das abas.
tabRegistos / tabVendas	JPanel	As duas abas.
tabelaRegistos	JTable	Tabela de produtos (uma coluna por Atributo).
modeloRegistos / modeloVendas	DefaultTableModel	Os dados das duas tabelas (só de leitura).
labelTotalQtd / labelTotalVendas	JLabel	Total de unidades vendidas e receita total (MT).
AoGravar	interface	Callback da janela de produto: gravar(dados, janela) devolve true para fechar.
Criterio	classe interna	Atributo + valor escolhidos numa janela de busca.

Construção do ecrã

Função	O que faz	Como
AplicacaoPadaria(root, perfil, aoTerminarSessao)	Inicia a janela principal.	Guarda o perfil, calcula eDono, carrega os dados de exemplo, constrói o layout, define tamanho e centra.
construirLayout()	Constrói a janela toda.	BorderLayout: barra de topo (norte), sidebar (oeste) e abas (centro).
construirBarraTopo()	Barra de topo.	Nome da padaria, saudação, perfil e botão "Sair".
saudacao()	Texto de boas-vindas.	Bom dia / Boa tarde / Boa noite conforme a hora, com o nome do utilizador.
terminarSessao()	Sair.	Pede confirmação, fecha a janela (dispose) e chama aoTerminarSessao.
construirSidebar()	Botões das operações.	Um LinkedHashMap texto → método. Cria um botão para cada um e salta os que estão em OPERACOES_RESTRITAS_AO_DONO quando o utilizador não é o dono.
construirAbas()	As duas abas.	Registos: JTable com uma coluna por Atributo. Vendas: JTable de vendas, totais e botão "+ Registar Venda".
popularExemplo()	Dados de exemplo.	Cadastra 4 produtos e regista 3 vendas.

Auxiliares

Função	O que faz	Como
<code>atualizarTabelaRegistos()</code>	Mostra todos os produtos.	Chama a versão abaixo com lista.
<code>atualizarTabelaRegistos(IntefaceGeraI produtos)</code>	Mostra os produtos de qualquer lista.	Limpa o modelo e acrescenta uma linha por produto com <code>tamanho()</code> / <code>pega(i)</code> . Serve para a lista principal e para os resultados das buscas.
<code>atualizarTabelaVendas()</code>	Redesenha a tabela de vendas.	Volta a preencher a partir de <code>listaVendas</code> e actualiza os dois totais.
<code>exigeDono()</code>	Verificação de permissão.	Se o utilizador não for o dono, mostra "Acesso restrito" e devolve <code>true</code> (quem chamou pára).
<code>novaJanela(titulo) / corpo(janela)</code>	Cria um diálogo.	<code>JDialog</code> modal com um corpo em <code>GridBagLayout</code> e margens de 24 px.
<code>mostrarJanela(janela)</code>	Mostra o diálogo.	<code>pack</code> , centra sobre a janela principal e <code>setVisible(true)</code> , que bloqueia até fechar.
<code>grelha(corpo, componente, linha, coluna, largura)</code>	Coloca um componente no diálogo.	<code>GridBagConstraints</code> com margens; rótulos à direita, botões centrados.
<code>janelaProduto(titulo, textoBotao, aoConfirmar, produto)</code>	Janela com os dados do produto (Cadastrar e Alterar).	Um campo por Atributo, com dicas. Com produto, vem preenchida e com o código bloqueado. Ao confirmar chama <code>aoConfirmar.gravar(dados, janela)</code> ; se devolver <code>false</code> a janela fica aberta.
<code>comboAtributos()</code>	<code>JComboBox</code> de atributos.	Mostra os rótulos ("Preço"), sem nenhum escolhido.
<code>pedirAtributoValor(titulo)</code>	Diálogo: escolher atributo e escrever valor.	Avisa se não houver atributo; devolve um <code>Criterio</code> ou <code>null</code> .
<code>pedirValorSimples(titulo, textoRotulo)</code>	Diálogo: escrever um valor.	Mesmo padrão com um só campo; devolve o texto ou <code>null</code> .
<code>info / aviso / erro(...)</code>	Mensagens.	<code>JOptionPane</code> com o ícone certo.
<code>lerInteiro(texto) / lerDecimal(texto)</code>	Converte texto em número.	Devolvem <code>null</code> se não for um número; <code>lerDecimal</code> aceita "12,5" e "12.5".

Acções (botões da sidebar)

Função	O que faz	Como
<code>acaoBuscarProduto()</code>	Busca por código.	Diálogo pede o código; <code>buscarPorCodigo</code> ; mostra só esse produto e selecciona a linha.
<code>acaoCadastrar()</code> [dono]	Novo produto.	<code>janelaProduto</code> vazia. Converte código/quantidade com <code>lerInteiro</code> e preço com <code>lerDecimal</code> , chama <code>lista.cadastrar</code> e actualiza a tabela. Números inválidos ou código repetido mostram erro e mantêm a janela aberta.
<code>acaoAlterar()</code> [dono]	Editar produto.	Pede o código, abre <code>janelaProduto</code> preenchida; campos vazios mantêm o valor; chama <code>alterarPorCodigo</code> .
<code>acaoBuscarUmAtributo()</code>	Busca por 1 atributo.	<code>pedirAtributoValor</code> + <code>buscarPorUmAtributo</code> ; mostra os resultados ou “Nenhum produto encontrado”.
<code>acaoBuscarDoisAtributos()</code>	Busca por 2 atributos.	Diálogo com dois pares atributo/valor; <code>buscarPorDoisAtributos</code> .
<code>acaoEliminarPosicao()</code> [dono]	Eliminar por posição.	<code>pedirValorSimple</code> ; <code>eliminarPorPosicao</code> ; mostra o nome do produto removido.
<code>acaoEliminarCodigo()</code> [dono]	Eliminar por código.	<code>pedirValorSimple</code> ; <code>eliminarPorCodigo</code> .
<code>acaoListarTodos()</code>	Mostrar todos.	Actualiza a tabela e selecciona a aba Registos.
<code>acaoListarCriterio()</code>	Listar por critério.	<code>pedirAtributoValor</code> + <code>listarPorCriterio</code> .
<code>acaoListarOrdenado()</code>	Listar ordenado.	Diálogo com <code>JComboBox</code> de atributo e caixa “Ordem decrescente”; <code>listarOrdenado</code> .
<code>registarVenda()</code>	Registar venda (sidebar e aba Vendas).	Pede código e quantidade. Verifica: inteiros positivos, produto existe, stock suficiente. Só depois desconta o stock, chama <code>listaVendas.registarVenda</code> , actualiza as duas tabelas e mostra o total.

5. Testes

`test/listas_duplamente_ligadas/TestesListas.java` tem 13 testes (cadastro, código repetido, buscas, alteração, eliminação nas pontas e no meio, esvaziar e voltar a encher, ordenação e totais de vendas). Não usa bibliotecas: corre como um programa normal e termina com o código 1 se algum teste falhar.

6. Limites conhecidos

- Ordenar por validade compara texto, por isso “01/12/2026” fica antes de “15/09/2026”. Solução: guardar a validade como `LocalDate`.
- Os dados perdem-se ao sair ou fechar. Solução: gravar num ficheiro e carregar no construtor de `AplicacaoPadaria`.
- Percorrer a lista com `pega(i)` recomeça sempre do primeiro nó, por isso um percurso completo custa $O(n^2)$ em vez de $O(n)$. Para uma padaria chega; para muitos produtos, a lista precisaria de um iterador.
- Buscas por código são $O(n)$. Solução para muitos produtos: um `HashMap<Integer, Produto>` ao lado da lista.
- Senhas em texto simples no código: aceitável só num trabalho académico.